Lab notes

What we tried while building this modpack, how we measured it, and why most of it did not ship.

The modpack is seven mods for Claude Code. Behind them is a lab where every idea has to show, in graded runs, that it saves something without making the work worse. Most ideas don't make it: a nudge Claude never followed, a gate that reassured falsely, a 10,000-star tool that made sessions more expensive. These notes are about those decisions. The data behind every number is in `benchmarks/`, with the harness to run it again.

> **Who wrote this.** Claude (Opus 5.5) wrote these notes from this repository's results files and commit history; HarzerHeribert reviewed them. We build a competing pack, so read what we say about other tools with that in mind: every claim about them links to their own words, and every number of ours to the runs behind it. If we got something wrong about your project, open an issue and we will correct it here.

How we measure

The same setup runs through every note unless it says otherwise.

- **The work.** Seven tasks on click 8.5.0 (79 files of Python), from a one-line lookup to a feature, later eight with a signature change, plus four tasks on a generated 54-file deployment repository (Kubernetes, Compose, GitHub Actions, Ansible). Every task is graded by a fixed rule: hidden tests for code, a parser for YAML, a term list for explanations. A run that saves money and gets the answer wrong counts as a failure, not a saving.
- **The runs.** Headless Claude Code (`claude -p`), Claude Opus 5.5, one fresh copy of the repository per run. Each condition names exactly which mods load, so nobody's personal settings decide anything. A *long session* forks a prepared session that has already read all of click's source, `src/click` (about 220k tokens of context), which is where real work spends most of its time.
- **The cost.** Read request by request from the transcripts and priced at list prices: Opus 5.5 at $4 per million input tokens, $20 output, $0.20 for a cache read, $8 for a cache write (Claude Code writes with the one-hour lifetime; a five-minute write is $5). On a subscription these are API-equivalent estimates, not a bill.
- **The uncertainty.** Two to eight runs per task and condition. Differences are reported for the average task with 95% bootstrap intervals, resampling runs within each task (Efron & Tibshirani 1993). An interval that crosses zero means *not shown*, and we say so. Before a confirmation run we wrote down what would count as confirmed (an example).
- **The limits.** Mostly one small Python repository and tasks every condition solves (note 9 repeats the main comparisons on TypeScript and on real bug reports): these benchmarks can show cost and step differences at equal quality. They cannot show where a weaker setup fails on harder work. Each note says what its data can and cannot carry.

References

- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *PNAS*, 115(11), 2600–2606.

The cheapest step is the one Claude never takes

Lab notes 1 · 15 Oct 2026 · written by Claude from this repository's data

Ask most people what makes a Claude Code session expensive and they will say tokens: big files, long answers, a chatty model. That is true, but it points at the wrong lever. Our first useful result was learning where the money actually goes.

Every step re-reads everything

A Claude Code session is a loop. Claude gets the conversation so far, decides on one step (read a file, run a grep, make an edit), the tool answers, and the answer joins the conversation. Then the whole conversation goes back to the model for the next step.

The whole conversation, every time. Prompt caching makes the re-read cheap per token: a cached token costs $0.20 per million instead of $4 on Opus 5.5. It does not make it free, and it multiplies. In a session holding 220,000 tokens, every step costs about $0.044 before Claude has thought about anything. A task that takes eight steps instead of five pays that three more times, plus the output of each step, plus whatever those steps add to the context, which every later step re-reads.

So the unit of cost is the **request**: a step of the main model. That shaped everything else. A mod that makes each tool answer 10% shorter saves a little. A mod that removes a step saves a whole re-read of everything.

What the pack actually saves

The pack's first measurements came out exactly along that line (SUITE.md). Tokens per request barely moved. Requests did: 5.2 per task with the pack against 7.2 without, on the same seven tasks, every run graded correct.

The release measurement of version 0.3.0, 116 runs, put numbers on it (SUITE.md):

	Change in cost per task, pack against plain Claude Code	95% interval
Long session (Claude has read the project)	**−26.2%**	−31.3% to −20.9%
Fresh session, multi-step tasks	−10.9%	−17.2% to −3.9%
Fresh session, quick questions	**+5.5%**	+1.2% to +10.2%

The long session is where the pack pays most, because that is where a removed step removes the most re-reading. In a long session Claude needed 3.5 requests per task with the pack instead of 5.4.

Which mod does it

An ablation takes one mod out at a time and measures what is lost (SUITE.md). The answer was not the mod we expected.

- **cells** lets Claude make several edits and run the tests in one step, through Claude Code's own edit tool. Taking it out made the pack 8.9% more expensive [+2.1%, +16.4%], and 16% on multi-step work. The feature task took 14.2 requests without it instead of 9.8.
- **lsp-plus**, which answers code searches from the language server, showed no net effect on the whole (−0.6% [−7.2%, +6.7%]). On the one task where it fires most, a rename, the pack without it needed 5.0 requests instead of 3.2. It helps exactly where Claude searches for a symbol, and is silent elsewhere.

The quick-question tax

The +5.5% on quick questions is the other side of the same arithmetic. The pack's tools, their descriptions and its guidance are about 4,000 tokens: 30.6k in the first request of a session instead of 26.7k. Cached, that is about $0.001 per request. But a fresh session has to write it to the cache first, and a three-step lookup has nothing to win back.

We could have hidden this in an average. We put it in the README instead, because it is the honest shape of the trade: the pack costs a little where there is nothing to save, and saves a lot where there is.

Then it did not replicate. Measured again two days later with the next version of the pack (note 8), quick questions cost **3.6% less** [−8.2%, +1.3%], and on a TypeScript library 9.9% less (note 9). Both days pooled: +2.3% [−1.1%, +6.0%], an interval across zero. The arithmetic above is real, but it is about a cent, and a cent is inside the spread of a three-step lookup from one day to the next. The README now says "about even". The long session replicated in the other direction: −39.9% on the second day, −33.2% [−38.9%, −26.9%] pooled.

The biggest saving was not a mod

While measuring that tax, we found a bigger one we were paying without knowing it. The person running this lab had six claude.ai connectors enabled (mail, a whiteboard, file storage and others), called between 0 and 13 times each in 946 sessions. Turning them off took the deferred tools offered to every session from 378 to 15, about 6,000 tokens of names and instructions, and cut the cost of the four quick tasks by 34%, with the same number of requests (SUITE.md). That is more than any mod in this pack saves on small work. If you take one thing from these notes: look at what your sessions carry that you never use.

What this changed

- We count requests first, tokens second. A feature that adds tokens but removes a step is usually a win; one that saves tokens but adds a step usually is not.

- We measure in long sessions, not only fresh ones. A fresh session is where demos happen; a long session is where the money goes.
- A mod has to pay for its own prompt. Anything that adds text to every request has to save more than it costs on the tasks where it does nothing.

Method & limits

Click 8.5.0, Opus 5.5, headless Claude Code, costs from transcripts at list prices; 2–8 runs per task and condition; intervals are 95% bootstrap intervals over runs. The connector measurement compared runs from different hours, so the cache's state may differ a little; the size of the effect is not in doubt. All tasks were solved by every condition, so these numbers say nothing about quality on harder work. Data: `results-030.jsonl`, `results-ablation.jsonl`, `results-noconn.jsonl`; harness: `bench.py`.

Sources

- Anthropic, Prompt caching: how cached prompt prefixes are written and read, and what each costs.

Notes nobody reads

Lab notes 2 · 15 Oct 2026 · written by Claude from this repository's data

The most natural way to improve an AI agent with a plugin is to give it advice. Notice it doing something slow, tell it the faster way. We built two mods on that idea. Both measurements came back with the same number: zero.

---

45 hints, 0 followed

When Claude searches code with `grep`, it gets every line that contains the text: definitions, calls, comments, strings, a variable that happens to share the name. A language server knows the difference. It can list exactly the functions that call `format_filename`, without the noise.

So lsp-plus watched for greps that looked for a code symbol and attached a short note to the result: *this is a code symbol; `LSP findReferences` on utils.py:442 lists exactly its uses*. Correct, specific, one call away.

In every benchmark run with the pack, that note fired 45 times. Claude acted on it 0 times (SUITE.md).

Looking at the transcripts, the reason is plain economics, the model's own. The note arrives after the grep has already answered. Following it means two more requests: one to load the LSP tool (it is deferred, one of hundreds of tools in those sessions), one to call it. Two steps to get an answer Claude believes it already has. From where the model stands, ignoring the note is the efficient choice.

Answers instead of advice

So we stopped advising and started answering. When a grep looks for a code symbol, lsp-plus now asks the language server itself, inside the hook, and attaches the server's answer to the grep's result: the callers, each named by its class and method, or the references by file (SUITE.md).

```
`resolve_color_default` per the language server: called from 5 functions in 3 files (code only: st
src/click/exceptions.py: ClickException.__init__ (48)
src/click/termui.py: _readline_prompt (99), get_pager_file (362), progressbar (581)
src/click/utils.py: echo (342)
```

No extra step for Claude: the answer is in the place it was already reading. On the rename task that cut requests by 25% and cost by 6.5%. Where no grep asked for a symbol, it said nothing and cost nothing. The note that asked Claude to do the same thing itself was switched off.

---

174 runs, 0 delegations

The second idea was bigger. Opus is expensive; Sonnet and Haiku are much cheaper. For well-specified work (implement this feature, write these tests), Opus could hand the work to a cheaper model and only

check the result. The delegate mod offers exactly that: a `hand_off` tool, with the evidence (the diff, the test run) coming back for Opus to judge.

Asked to hand off, it works: a feature cost 27–37% less, every result correct. The question was whether Opus would hand off without being asked. We tried everything we could think of (SUITE.md):

What we tried	Runs	Hand-offs
The tool offered	24	0
The tool offered, with its measured savings in its description	4	0
A routing rule in the system prompt naming the work to hand off	14	0
Claude Code's whole system prompt replaced by one sentence and that rule	4	0
The person ending every prompt with "Delegate where you see fit."	28	0
The same tool described as an ability, `implement_spec`, with no word of delegation	8	0
Telling Opus what each of its requests costs in this session	4	0
Claude Code's own Agent tool, available in every run	174	0

It is not the harness: with Claude Code's system prompt gone, nothing changed. It is not permission: the person's own words in the prompt changed nothing. Opus 5.5 does not give away work it can do itself.

On small work that is the right call: a hand-off takes two to five requests of its own, so a small question handed off costs up to three and a half times as much. Opus only misses the big, specified work.

An order works

One thing did work, every time. When a mod attached an order to the prompt, phrased as the person's ("The person approved this plan; implement it through hand_off"), Opus handed off in 8 of 8 runs, and the feature cost 27% less (SUITE.md).

That became the design. When you approve a plan of 20 lines or more in plan mode, delegate gives that order: the plan becomes the worker's spec. Claude does not decide to delegate; your approval does.

Other tools found the same thing

We are not the only ones who learned this. The code-graph tool Trail-Code (formerly graft) injects an instruction to "reach for trail first" at session start, and with prompts adds pointers or a reminder to run `trail ask` "before grepping". Its own source has a comment from an investigated session: *"the agent did not pull. It grepped 38 times."* (`src/claude/format.ts`). Its maintainer, on Hacker News: *"claude never actually uses them as it's trained to use grep"* (HN). graphify, another code-graph tool, injects a note before searches: *"MANDATORY: … You MUST run `graphify query "<question>"` before grepping raw files"* (`graphify/cli.py`).

Three projects, the same observation: an agent trained to grep keeps grepping. Shouting does not change that. Answering inside the grep does.

What this changed

- A mod that wants Claude to behave differently must change what Claude sees in the place it already looks, not ask it to look elsewhere.
- Advice is not free. Every note costs tokens in every later request; a note that is ignored is pure cost. Our rule since: a mod speaks only where it has proof that speaking helps.
- When a choice really is the person's (delegating, spending), make the person's action the trigger, not a model's judgment the model has shown it won't make.

Method & limits

Click 8.5.0, Opus 5.5, headless Claude Code; hand-off counts read from transcripts. "0 hand-offs" holds for these tasks and this model version; a future model may behave differently, and the harness to check is public. The 45 notes are all the notes in all pack runs up to the quiet pack, across conditions. Data: `results-quiet.jsonl`, `results-answer.jsonl`, `results-prompt.jsonl`, `results-fit.jsonl`, `results-delegation-ideas.jsonl`.

Sources

- trailhq, Trail-Code, source at `ead585b` (checked 15 Oct 2026).
- Show HN discussion of graft, maintainer comments.
- Graphify-Labs, graphify, `graphify/cli.py` (the hook-guard note).

A 10,000-star code graph on our bench

Lab notes 3 · 15 Oct 2026 · written by Claude from this repository's data

A friend asked a fair question: why build our own code tools when projects like graphify (about 127,000 GitHub stars) and Trail-Code (about 10,000) put a whole codebase into a graph that Claude can query, and report large savings doing it? Someone they knew had cut their Claude usage sharply with one of them.

So we put one on our bench. This note is about what happened, and about how much a benchmark like ours can and cannot say about a tool built for a different job.

Two tools, one idea

Both tools parse a repository with tree-sitter into a graph of definitions and calls, and hand Claude ways to query it. Neither needs a language model to build the code graph, so an existing project gets one right away; that was one of the myths we had going in.

They differ in the thing that turned out to matter most: **keeping the graph current**. graphify's Claude Code setup asks the model, in CLAUDE.md, to run `graphify update .` after changing code, or relies on opt-in git hooks and a file watcher; its issue tracker has reports of answers from graphs days old (#2216, #3718). Trail-Code checks the working tree before every query (a few milliseconds) and rebuilds what changed, with no model involved (`src/graph/refresh.ts`). We benchmarked Trail-Code, the one whose design does not depend on Claude remembering a chore.

What they claim, and what others measured

Trail-Code reports a 162-run sweep with Claude Sonnet 5 answering questions about two repositories: 32% lower cost, 46% fewer tool calls, the same correctness (93% vs 93%), judged by another model (README). It also reports SWE-bench Verified, 27 against 33 resolved out of 50; in the Hacker News discussion a commenter put that difference at p≈0.22, and the maintainer explained the growing sample size by usage limits running out (HN).

graphify's documented benchmark is "71.5x fewer tokens per query" on a 52-file corpus (docs/how-it-works.md). Its own benchmark script compares one query's output against reading **the whole corpus**, and the same page reports about 1x on a 6-file corpus. That is a ceiling, not what an agent that greps and reads a few files would pay.

Two independent checks stand out, both from users in Trail-Code's own issue tracker:

- #117: on 186 cases built from merged pull requests in four repositories (graft 0.10.1), the tool's ranking put 48.5% of the files each pull request changed into its top 10, against **66.5% for plain BM25 keyword search over whole files**. A contributor later proposed adopting whole-file BM25 (#257); the maintainers declined after their own re-test found their current ranking as good or better.

- #506: on a 291-file Swift/Kotlin repository, the context packs it returned were 2.5–4 times larger than `rg` plus a targeted read, and its top hit was right in one of three questions.

Our setup

- Trail-Code 0.22.1 from npm, wired with its own `trail init` (skill, hooks, MCP server, its graph of click: 2,028 nodes and 3,221 edges from 79 files, built in about a second). We kept only the repository-level wiring, so its hooks ran once (#497 describes them running twice when both levels are installed), started its MCP server from the installed version, and turned its telemetry off.
- **Both arms had our pack loaded.** This measures what Trail-Code adds to the pack, not Trail-Code against plain Claude Code. That is the question a pack user has, and it is the harder test for Trail-Code: lsp-plus already answers symbol searches.
- Opus 5.5, click 8.5.0, eight tasks fresh (four runs each) and seven in a long session that had already read the code (two runs each). Every result graded by hidden tests or fixed rules.

What we measured

	Trail-Code added to the pack	95% interval	Runs
Fresh session	**+12.8%** cost	+7.5% to +18.3%	64
Long session	**+32.1%** cost	+21.9% to +43.6%	28
Fresh, without its learning hook	+4.0% cost	+0.1% to +8.0%	64

It cost more on every one of the eight tasks, fresh and long. Quality was the same: every answer correct.

Where the extra cost came from

A learning note that ate the answer. Trail-Code's end-of-turn hook stops Claude once per session to write a "team learning" into the repository. On the explanation task, in all four fresh runs, Claude explained how click resolves an option's value, and then the hook made it write a learning file, so its final message was about the file, not the answer. The note ended with a line for a contributor named from the repository's history: *"Commit what you learned and their agents get to a PR 3x faster"* (`src/notes/notes.ts`). In an interactive session you would see both messages; in a headless run the final message is the result. We graded Trail-Code fairly on all of Claude's messages (all correct), and measured the hook separately: it accounts for about 8 points of the cost [+4.0%, +13.5%], and 2–4 extra requests on that task. The learning file's name also contained the git user's name.

Context Claude did not need. Without the learning hook, Trail-Code still cost 4% more. Its hooks put an orientation into the session and pointers into prompts, and every later request re-reads them. In the long session, where Claude had already read every file, that added context repeated what it knew, and the cost difference grew to 32%.

Tools Claude rarely reached for. Claude used Trail-Code's command line in 11 of its 32 fresh runs, mostly on the lookup and explanation tasks, and its six MCP tools in none. On the rename, the feature and the tests it worked as without it. That matches what Trail-Code's own maintainer wrote (see note 2), and what we saw with our own hints.

What this does and does not say

Our bench is a 79-file Python library, and our tasks mostly name what they are about: `format_filename`, `ParamType`, an option's default. Opus with the pack finds such things in two or three steps. That is the situation where a pre-built map has least to add, and the situation our users are in most of the time.

It says nothing about the job these tools are designed for: a large, unfamiliar repository where the question comes before anyone knows a symbol to search ("where does authentication happen?"). We did not test that, and the one independent test of that step we found (#117) favoured plain keyword search over the graph's ranking of that version. If you maintain one of these tools and think our setup is unfair to it, the harness is in `benchmarks/` and we will run what you suggest.

We measured it once more on real bug reports, where it reports its own SWE-bench results: 30 issues, again more expensive than the pack alone (+10.8% [0.0%, +25.5%]; note 9). What we took from these tools is in the next note.

Method & limits

Arms: Z (the pack as committed), T (the pack and Trail-Code as `trail init` wires a repository), TG (as T without Trail-Code's Stop hook), PL and TL (Z and T in a prepared 220k-token session). The four explanation runs of T score 0, 0, 0.33 and 0 in the results file, graded on the final message as the bench does; graded on all of Claude's messages, from the transcripts that stay in the lab, all four score 1.0. Costs from transcripts at list prices; intervals are 95% bootstrap intervals over runs within tasks. Four runs per task fresh, two in the long session: per-task differences are within the runs' spread; the totals' intervals are not. Data: `results-trail.jsonl`; setup: `trail_init()` and `TRAIL_ARMS` in `bench.py`.

Sources

- trailhq, Trail-Code (formerly graft), README, source and issues #117, #257, #497, #506; checked 15 Oct 2026.
- Graphify-Labs, graphify, docs/how-it-works.md, issues #2216, #3718; checked 15 Oct 2026.
- Show HN: graft, discussion of its SWE-bench sample.
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*.
- OpenAI (2024). Introducing SWE-bench Verified.

Borrowing the good ideas

Lab notes 4 · 15 Oct 2026 · written by Claude from this repository's data

The code-graph tools in the last note cost more on our bench than they saved. That does not make their ideas wrong. Two of them are good, and we tried to carry them over in a form that fits how Claude actually works: answers placed where it already looks, from sources that are never stale.

Idea 1: a pointer before the first search

Trail-Code's prompt hook adds a few likely places to a prompt before Claude starts, so it can skip the first search. The step before you know a symbol is the one a language server cannot help with, because it needs a symbol to ask about.

Our first version failed in calibration. We started with what the independent evaluation found strongest (Trail-Code #117): BM25 keyword ranking over whole files (Robertson & Zaragoza 2009), term counts from `git grep`, so there is no index to keep fresh. It speaks only when the best file is clearly ahead of the field.

Before spending a single benchmark run, we looked at what it would attach to 13 prompts: our seven task prompts and six everyday ones ("commit and push this", "run the tests and tell me what fails", "fix the typo in the README"...).

- The first version pointed "commit and push this" at an example program that happens to define `commit()` and `push()` functions.
- After teaching it to skip paths and flags, to weigh code over prose, and to need a rare code identifier from the prompt, it went quiet on 11 of 13 prompts. On the other two it pointed at the wrong code: once at the word `inner`, taken from a formatting example in the prompt ("nested functions as `outer.inner`"), and once at `pytest`.
- Silent where it should speak, confidently wrong where it spoke. A pointer that is wrong costs more than no pointer: Claude reads the wrong file first.

What survived is the deterministic half. When a prompt spells out a code name (`format_filename`, `ParamType`, `echo()`), scout looks up where that name is defined, with one `git grep` per name, and attaches the definition if there are one to three of them in code files (the code's own ahead of tests and examples):

```
Where the code names in this prompt are defined (git grep):
`resolve_color_default`: src/click/globals.py:54 `def resolve_color_default(color: bool | None = N
```

On the same 13 prompts it spoke on 4, correctly each time, and said nothing on the other 9. A name that is defined nowhere (a new one, a word in backticks) or in many places gets no pointer. Neither does a file Claude has already read, nor anything in a session past 80k tokens of context, where Claude has most likely read the code.

One caution we owe the reader: we calibrated on the same prompts we then benchmarked on. The surviving rule has no threshold we tuned, which limits the risk, but it is not a held-out test.

Idea 2: the blast radius after an edit

Trail-Code prints the callers of what you just edited, from its graph. Our version asks the live language server, and only when an edit changed a function's parameters, the moment callers may break:

```
`resolve_color_default`'s parameters changed; per the language server it is called from 5 places :
src/click/exceptions.py: __init__ (48)
src/click/termui.py: _readline_prompt (99), get_pager_file (362), progressbar (581)
src/click/utils.py: echo (342)
```

To give it a fair chance we added a task built for it: make `resolve_color_default`'s parameter keyword-only and rename it, so three positional callers must change, checked by a hidden test and a scan for stale calls.

What it measured

96 runs of the ideas, all correct (`results-ideas.jsonl`), beside the pack's own runs. The first round left every interval across zero, so we ran scout against no scout again on the five tasks where it speaks, for eight runs per task:

	Change in cost	95% interval
The pack with both ideas, against the pack, 8 tasks	−2.5%	−6.9% to +2.0%
Taking scout out, on the 5 tasks where it speaks	+3.8%	−1.9% to +10.3%
Taking blast out, on the signature task	+3.7%	−2.0% to +11.0%

On the signature task itself, Claude needed 4.2 requests with the pack, 3.4 with both ideas, 5.2 with only the blast radius, and 6.5 with Trail-Code.

Twice the runs, the same answer: a small saving in the direction we hoped, never a wrong result, and an interval that still crosses zero. By our own rule that is not enough to ship. We measured once more on 30 real bug reports, whose text names functions far more often than our prompts: scout spoke on 20 of them and saved nothing (+7.3% against the pack [−6.7%, +20.7%]; note 9). Both ideas stay out of the pack.

Why the blast radius barely matters here was visible in the transcripts: before editing a function, Claude greps for it, and lsp-plus already answers that grep with the same list of callers. By the time the signature changes, Claude knows who calls it. An idea that is valuable in a tool without lsp-plus is redundant in a pack with it. It stays in lsp-plus, off by default.

What this changed

- Calibrate before you benchmark. Thirteen prompts and a script cost nothing and stopped a feature that would have spent money being wrong.
- Prefer the deterministic version of an idea. "Where is the name you typed defined" can be wrong only if the code changed in the last second; "which file is most similar to your words" can be wrong in a hundred ways.
- An idea's value depends on the pack around it. Measure it as an addition to what already ships, not on its own.

Method & limits

Arms: Z (the pack), N (the pack with scout and lsp-plus's `blast` on), N-scout (without scout), N-blast (without blast). Opus 5.5, click 8.5.0, fresh sessions; four runs per task and arm, eight for N and N-scout on the five tasks where scout speaks. Scout's calibration script and prompts are in the lab. Data: `results-ideas.jsonl`, the pack's runs in `results-trail.jsonl`.

Sources

- trailhq, Trail-Code: prompt hook and post-edit blast radius; issue #117.
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.

The graveyard

Lab notes 5 · 15 Oct 2026 · written by Claude from this repository's data

Six mods ship in this pack. Many more ideas were built, measured and dropped. A project's README usually shows the survivors; this note shows the rest, because the reasons they failed are more useful than the features that worked. Each grave has the idea, why it looked good, and the number that ended it.

The checklist that said everything was fine

lastenheft (German for a requirements specification) turned an order into a list of checkable requirements and refused to let Claude stop until each was proven. Agents that report "done" on half-finished work are a real problem; a gate against it looked like obvious value.

The first batch looked promising. On six planned tasks from a private research codebase, with Claude Haiku 5.5, the gate lowered "said done while incomplete" from 92% to 75% of runs, a paired difference of −17 points [−33, −4].

We did not ship it. Seven measures had been compared across three conditions, so one good-looking interval was a lead, not a finding. Before running more, we wrote down the one measure and the rule that would count as a confirmation (preregistration; Nosek et al. 2018), then ran the same tasks with fresh repeats:

- Confirmation: 88% against 92%, **−4 points [−12.5, 0.0]. Not confirmed.** Further repeats gave the same −4. The first batch's −17 had been mostly luck, the shrinkage you expect when the first estimate is picked because it looked large (Ioannidis 2008).
- The gate also cost about 20 more tool calls per run.
- A lighter version, a checklist without the gate, came out the same: −4 points [−14.6, +6.2], not confirmed. And it showed why: in **38 of 43 runs that were incomplete, every point on the list showed passed.** Claude's own tests passed; they just did not test what was missing. A checklist that turns green on incomplete work does worse than nothing, because it reassures the person.

lastenheft stays in the lab, switched off. (Its data stays private: the tasks come from a private codebase.)

The hint nobody followed

lsp-plus's grep note told Claude which language-server call answers its search exactly. **0 of 45** followed. The story, and what replaced it, is in note 2. The note is off by default.

Four ways to make Opus delegate

A rule in the system prompt naming the work to hand off (**0** hand-offs in 18 runs), the person's own "Delegate where you see fit" (**0 of 28**), the tool presented as an ability, `implement_spec(spec, check)`,

with no word of delegation (**0 of 8**, and the unused tool cost 11–18% to carry), and telling Opus what each of its requests costs (**0 of 4**). Only an order works (note 2).

Handing off the small stuff

If handing off a feature saves a third, why not hand off lookups? We let delegate take read-only work. Told to hand off a lookup, Opus turned it into an edit task (the worker wrote the answer to a file, the check printed the file) and the runs cost **13–33% more**, leaving stray files in the repository. A hand-off takes two to five requests of its own; a lookup takes one. Reverted.

The projection that was wrong

An early analysis projected that hand-offs save more the fuller the session, **−53%** for the feature at 200k tokens, by assuming each step re-reads more. We measured it in a 233k-token session: the feature saved **37%**, about what it saved fresh, and a small question handed off cost up to **3.6 times** as much (the lookup 2.5 times) (SUITE.md). The projection assumed Opus needs as many steps in a session that knows the code as in a fresh one. It needs far fewer.

Moving text before the cache boundary

cells' guidance is the same text in every session. In theory, putting it in a system-prompt section shared across sessions means writing it to the cache once and reading it ever after. Measured: **+1.0% [−5%, +8%]**, no saving; the first request wrote no less. Reverted.

Keyword pointers

BM25 ranking to point Claude at files before it searches. It failed calibration before a single benchmark run: silent on 11 of 13 prompts and wrong on the other 2. The version that survived is deterministic (note 4).

The log squeezer

squeeze replaced a long test log with its failures (file, line, message) and kept the full log on disk. Locally it is excellent: all 18 failures of an 18-failure run kept, 85–99% of the text gone, in about a millisecond. On a task with 18 failing tests it fired **0 times in 4 runs**, because Opus piped every test run through `tail` or `grep` itself, 24 times across the 8 runs. We tried Claude Haiku 5.5, expecting a smaller model to dump whole logs: it piped through `tail` too (44 times in 8 runs), and squeeze fired 0 times again. A tool for a habit neither model has.

Compression that forgets

ScaleDown's Compress keeps the passages that look relevant to a question. Asked five factual questions about click from a compressed copy, a model answered 4 of 5 when each question had its own compression, and **0–1 of 5** when one compression had to serve all five; one answer was confidently wrong (exit code 1 instead of 2) because the line that said 2 was gone. In a coding session the next question is never known in advance (note 7).

The mod nobody measured

turn-savers (lighter thinking on routine steps, the code at a failure attached to the failing run) was built and never measured. So it does not ship. A mod without a measurement is a guess with a version number.

What the graveyard teaches

- **The first good number is usually too good.** Write down what would confirm it, then run it again.
- **A check that passes is not a check that checks.** Agents' own tests share their blind spots.
- **Models have habits.** They grep, they tail logs, they keep work they can do. A feature that fights a habit loses; one that answers inside it wins.
- **Removing is a result.** Every grave here made the pack cheaper or simpler.

Method & limits

Delegate bench results: `benchmarks/delegate-bench/` and SUITE.md. The lastenheft runs used Claude Haiku 5.5 on tasks from a private codebase; the preregistration texts and analysis stay in the lab with them. squeeze: `results-squeeze.jsonl`. ScaleDown measurements: note 7.

Sources

- Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *PNAS*, 115(11), 2600–2606.
- Ioannidis, J. P. A. (2008). Why most discovered true associations are inflated. *Epidemiology*, 19(5), 640–648.

One server per extension

Lab notes 6 · 15 Oct 2026 · written by Claude from this repository's data

lsp-plus answers Claude's code searches from a language server. That only works where a server answers. The obvious next question, asked by someone who works with infrastructure all day: what about Terraform, Ansible, Kubernetes YAML? This note is about adding languages, and how often "there is a language server for that" turned out to be the start of the work, not the end.

The rule that shapes everything

Claude Code picks a language server by file extension, and only by extension: no file patterns, no first-line detection. Each extension gets exactly one server; when two plugins claim the same one, the first registered wins (plugin docs). For most languages that is fine. For `.yaml` it is a problem, because Kubernetes manifests, Compose files, GitHub Actions, Ansible playbooks and Helm values all share it.

Five languages, five surprises

We gave lsp-plus Go, Kotlin and Ruby (whose servers it could install but whose files its grep answer ignored), and declared servers for shell scripts and Terraform. Then we tested each against a real server, in a real Claude Code session, on a small repository where we knew the right answer.

- **Go** worked at once. gopls even found callers through method receivers (`func (c *Calculator) Total`).
- **Shell** worked after one change. bash-language-server has no call hierarchy, so a search for a function got no answer. lsp-plus now falls back to the server's references when there is no call hierarchy.
- **Ruby** crashed with exit code 78 before answering anything. ruby-lsp refuses to start in a project with a `Gemfile` but no `Gemfile.lock` . Most apps commit the lock file; many gems do not. Now in the setup guide.
- **Terraform** answered symbol searches but found no references. Our own LSP client got the same, so it was the server, not Claude Code. Its log said why: no provider schema for `hashicorp/aws` . terraform-ls only sees references inside blocks whose schema it knows, and the Homebrew build of terraform-ls ships without the provider schemas HashiCorp's own build embeds (0 schema entries in the binary against 6). Installed from HashiCorp's tap, every `var.region` was found.
- **Kotlin** we could not test: JetBrains' Homebrew formula pointed at a download that returned 404 that day. It is covered by tests only.

Terraform also needed lsp-plus to understand its names. A grep for `var.region` or `aws_s3_bucket.logs` searches for an address, not an identifier. lsp-plus now maps an address to the block that declares it, asks the server in each module (directory) where the grep found it, and answers with the references:

```
`var.region` per the language server: 4 references in code, in 3 files (code only: strings, commer
main.tf: 2
outputs.tf: 2, 6
providers.tf: 2
```

Ansible and Kubernetes: no references to give

For YAML the research came first, and it was sobering (sources):

- The **Ansible language server** offers hover, completion, definition and diagnostics, but no references and no symbols; its "definition" jumps from a module name to the module's Python source. A pull request adding references for handlers, variables and roles was closed unmerged. And it decides a file is Ansible by path patterns and content, which Claude Code cannot express.
- **yaml-language-server** with Kubernetes schemas validates structure (a misspelled field, a string where a number belongs) but has no references: its "definition" only jumps from a YAML alias to its anchor in the same file.
- No language server we found resolves the names that actually connect a Kubernetes deployment: a Service's selector to the pods it selects, a `configMapRef` to the ConfigMap, a RoleBinding to its ServiceAccount.

So the question became: one YAML server for everything is forced on us anyway; could it also check the names?

---

yaml-plus

yaml-plus is one server for every `.yaml` file. It runs yaml-language-server underneath for syntax and schemas, choosing the schema from each file's content (Kubernetes by `apiVersion` and `kind`, Ansible by structure). It skips Helm templates, which are not YAML until rendered. On top it indexes the repository's names and checks the references between them:

- **Kubernetes**: ConfigMaps, ServiceAccounts and volume claims a pod uses; Services and the workloads they select; Ingress backends, autoscaler targets, role bindings; kustomize paths.
- **Compose**: `depends_on`, networks, volumes, configs, secrets.
- **GitHub Actions**: `needs`, `needs.X.outputs`, `steps.X`, inputs, local actions.
- **Ansible**: handlers and `notify`, roles, included task files.

It also answers go-to-definition and find-references for those names.

Its rule is to say nothing it is not sure of. It never flags a Secret (they are often created outside the repository), never a name from a remote base, never a variable. Run over about 30 local repositories and 6 public ones (Argo CD, kube-prometheus, the Kubernetes examples, awesome-compose, an Ansible book, VS Code), it raised one warning in 5,041 name uses: a PersistentVolumeClaim whose manifest exists only inside a README. It indexes 4,000 manifests in about 0.7 s.

One thing we did not expect. In a live session, yaml-plus's warning reached Claude after an edit:

> ConfigMap "cart-confg" (namespace shop) is not declared in this repository (did you mean "cart-config"?)

Reading the same broken file delivered nothing. Claude Code syncs a file to its language server when it is edited or asked about, not when it is read. A checker that only speaks after edits cannot help Claude *find* a broken reference, only avoid making one. yaml-plus therefore gained hooks: a read of a YAML file gets that file's name problems, and a grep for a name that hits YAML files gets where it is declared and used, the way lsp-plus answers code searches.

What it measured

We generated a 54-file deployment repository (ten services across Kubernetes, Compose, two GitHub Actions workflows and six Ansible roles) and four tasks:

- a ConfigMap rename across 11 references;
- six planted broken references, one in each kind of file and two more in Kubernetes;
- "which objects refer to this ServiceAccount";
- a Compose change.

Every task was graded by parsing the YAML, never by the server under test.

Model	yaml-plus added to the pack	95% interval	Runs	Correct
Opus 5.5	+2.0% cost	−1.3% to +5.3%	64	all
Haiku 5.5	about the same, at roughly one cent a run		32	all

yaml-plus spoke 56 times in Opus's runs. Opus found all six broken references and completed the 11-reference rename without it. We expected Haiku, a weaker model, to miss what the checker catches; it missed nothing either.

Fifty-four files are few enough for any model to read whole, and a checker cannot save what a model never misses. The question yaml-plus exists for is different: do names resolve correctly every time, and faster, in a repository too big to read?

So we made one. The same generator with 300 more services comes to 954 YAML files, and we hid ten broken references in it: the six from before and four among the generated services (a misspelled ConfigMap, a ServiceAccount spelled with the letter O for a zero, a Service whose selector matches no pods, an autoscaler aimed at a Deployment that does not exist).

Model	Without yaml-plus	With yaml-plus
Opus 5.5	10 of 10 in every run, $0.61, 10.2 requests	10 of 10 in every run, $0.70, 10.0 requests
Haiku 5.5	10 of 10 in every run, $0.03	10 of 10 in three runs, 8 in one, $0.03

Four runs per cell. Opus did not read 954 files. It listed them, read one of each kind, and then **wrote its own reference checker**: about 50 lines of Python with PyYAML, which it ran over the whole repository, and confirmed the result with `kubectl kustomize`. In one step, it built for this task the tool we had built for every task.

That is the honest end of this experiment. Within the time we could give it, we did not find the repository where yaml-plus earns its place: small ones are read whole, large ones get a checker written on the spot. The server costs no tokens until it speaks, only memory (about 200 MB for 4,000 manifests, plus yaml-language-server), so it does no harm to keep. But nothing here says it helps, and by our rule that keeps it out of the pack. A real infrastructure repository with irregular names, Helm charts and kustomize overlays, too tangled for a 50-line script, is the test it would still need.

Method & limits

Language checks: one headless Claude Code session per language on a small repository with known answers, the hook's note read from the transcript; terraform-ls also queried directly with a minimal LSP client. yaml-plus: 44 server tests and 13 hook tests, and the false-positive sweep described above, run locally. The YAML benchmark's generator and grader are in `yaml_repo.py`; runs in `results-yaml.jsonl`, `results-yaml-b.jsonl` and `results-yaml-haiku.jsonl`; the 954-file repository in `results-yaml-big.jsonl` and `results-yaml-big-haiku.jsonl`.

Sources

- Claude Code, Plugin components: LSP servers.
- HashiCorp, terraform-ls, `docs/schema.md` (provider schemas bundled at compile time in HashiCorp's released binaries).
- Red Hat, ansible-language-server, and the closed pull request #2647.
- Red Hat, yaml-language-server.
- Ruby LSP; bash-language-server; gopls.

Small models as plumbing

Lab notes 7 · 15 Oct 2026 · written by Claude from this repository's data

If every token in Claude's context is paid for again on every step, an obvious trick is to shrink what enters it. A long test log, a big diff, a sprawling grep result: hand it to a small, fast model that compresses or summarizes it, and give Claude the short version. We tested that with ScaleDown, an API with four modes (Compress, Summarize, Extract, Classify) and latencies under a second for small inputs.

The price question settled itself. The person running this lab paid €2.50 for 50 million tokens, about $0.054 per million: a quarter of what Claude charges just to *re-read* a cached token. Every call we made cost under 1% of what the same text would cost in Claude's context. So price is not the question. Whether the short version keeps what Claude needs is.

Compress: the right passages for the wrong question

Compress keeps the passages it judges relevant to a question and drops the rest. On click's source it kept 1–5% of the text, in 1.6–3.6 s. We asked five factual questions about click, with answers checked in the source, from the original text and from compressed copies:

Text given to the model	20k-token context	60k-token context
Original	5 of 5	5 of 5
Compressed once per question	4 of 5	4 of 5
Compressed once for all five questions	**0 of 5**	**1 of 5**

The miss in the per-question case was the same both times: the line of `confirm()` that shows `[Y/n]` had been cut. In the shared case one answer was confidently wrong: an error class "inherits exit code 1", because the line setting `exit_code = 2` was gone while its docstring stayed.

An agent's next question is never known when its context is built. Compression that keeps what matters for *a* question is compression that drops what matters for the next one. Prompt-compression research gets its best ratios by making compression question-aware (Jiang et al. 2024); the other side of that is a result that fits the one question it was made for.

Summarize: confident, and wrong about the important part

On a test log with three failures, Summarize named the three failing tests every time and **never** kept a file and line number. On a log with 18 failures, four of five summaries were just the counts line, every failure gone.

The worst case was a report from a worker model (the kind delegate hands back). It truthfully listed a diff and a test run with three failures, and then said, falsely, *"the new tests pass"*. A plain summary

repeated the false claim **3 of 3 times**, and once did not mention the failure at all. Told what to keep ("failing tests, exit codes, claims that contradict the check"), it kept the failures and flagged the false claim every time. Which raises the obvious question: if you have to tell it what matters, why not extract what matters yourself?

Extract: right, and slower than a parser

Extract with a schema (failing test, file, line, message) got the test logs right: all failures, all locations, every time. It took 4–10 s, and one call failed after 77 s. The local parser we had already written for the same job gets the same logs fully right in about a millisecond, with nothing leaving the machine. (On a large diff, Summarize recovered at most 41% of the changed functions and Extract at most 26%, where reading the diff's own hunk headers gets 80%.)

Classify: the one job it did perfectly

Classify sorts a text into labels you give it. We gave it the 8 benchmark prompts and 17 everyday ones and seven labels: lookup, explain, small edit, rename, feature, tests, other. Asked one prompt at a time: **25 of 25** right, median 0.42 s, $0.00003 a call. (In one batch call of all 25 it got one wrong, though the documentation says calibration is the same as for single calls.)

That is exactly the missing piece from note 2: Opus never hands work to a cheaper model unless told, and an order from a mod is followed every time. A classifier that recognizes "implement this feature" or "write tests for this" in 0.4 s could give that order for exactly the work where a hand-off pays (27–37% less on features and tests), and stay silent on small questions, where a hand-off costs up to 3.6 times as much.

So we built it. **route** is a small mod: when you send a prompt, it asks Classify what kind of work it is, with the same seven labels and rubrics that scored 25 of 25, and waits up to three seconds. If the answer is feature or test work with a score of at least 0.7, it attaches delegate's order beside the prompt: *the person's setting for feature work is: implement it through the hand_off tool*. Anything else, or no answer in time, adds nothing.

On our eight click tasks, four runs each, all 32 correct (`results-route.jsonl`):

	The order given	Hand-offs	Cost, the pack → the pack with route
The feature	4 of 4	4	$0.61 → **$0.41** (9.5 → 5.2 main requests)
The coverage task	4 of 4	4	$0.67 → **$0.36** (8.5 → 4.2)
The six other tasks	0 of 24	0	about the same

Every routing decision was right: the order on all eight feature and test prompts, on none of the other 24. On the two tasks it routed, the pack with route cost **39.6% less** than the pack alone [−44.8%, −33.7%]; on the other six, +4.3% [−0.1%, +9.0%], within the runs' spread. Over all eight, **−17.0%** against the pack [−20.5%, −13.1%] and **−21.7% against plain Claude Code** [−24.9%, −18.2%].

This closes the loop from note 2. Opus never delegates on its own and always follows an order; a 0.4-second classifier decides when an order is worth giving. What it costs: a ScaleDown key, about 0.4 s (up to 3 s) before each prompt starts, $0.00003 a prompt, and the prompt's text leaving the machine. So it can only ever be opt-in.

What it adds up to

Use	Verdict	Why
Compress Claude's context	No	Loses facts the next question needs
Summarize logs or worker reports	No	Drops locations; repeats false claims unless told what to keep
Extract failures from logs	Not needed	A local parser is as right, thousands of times faster, and private
Classify prompts for routing	**Yes, opt-in**	25 of 25, 0.4 s; routing feature and test work to a cheaper worker: −17% over the pack

Small models make good plumbing where the job is narrow and checkable: a label from a fixed set, on which a stronger model then acts. They make poor editors of what a stronger model will reason over, because they cannot know what it will need.

Privacy is part of the cost: every call sends the text to a processor in the US. ScaleDown's data processing terms, as provided with the account, promise zero retention and no training on customer data. We sent only public code (click) and text we generated, and any mod that used it would have to be off by default and say so.

Method & limits

151 API calls, 1.8 million billed tokens in total, on click's source and on logs, diffs and greps generated from it; answers scored against ground truth worked out from the code. Fidelity of compressed context judged by Claude Haiku 5.5 answering from the text alone. Five runs per case unless noted; Classify on 25 prompts, one run each plus repeats. The key ran out of credits midway through the first round; the second round ran on new credits. Scripts and results stay in the lab with the API client.

Sources

- ScaleDown, documentation, model overview and Classify reference; price and data terms as provided with the account.
- Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., & Qiu, L. (2024). LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. *ACL 2024* (Vol. 1), 1658–1677.

Our plain runs were not plain

Lab notes 8 · 15 Oct 2026 · written by Claude from this repository's data

Every comparison in these notes rests on one thing: knowing exactly what loaded in each run. On 15 October we found out that for a day and a half, in one condition, we had not known it. This note is about how that happened, how close it came to becoming a finding, and what we changed.

The good news that wasn't

The bench names what loads in each condition. Our mods are listed in an environment variable; "plain Claude Code" is the same list, empty. That had worked for hundreds of runs.

On the afternoon of the 15th we re-ran plain Claude Code on the click tasks, to compare the pack against today's version of Claude Code rather than the one from two days earlier. The result was striking: plain cost the same as the pack (+2.1% for the pack [−2.4%, +7.0%]). The feature task, which plain had needed 14.8 requests for, now took 9.2.

There was a ready story: Claude Code had caught up. A new version, smarter defaults, the pack's advantage gone. It would have made a good note.

Before writing it, we looked at what plain was doing differently, request by request. In the feature runs, plain Claude Code called `mcp__cells__run_cell` five times. That tool exists only in our cells mod.

What had happened

On 14 October at 00:33, the person running this lab installed the released modpack from its marketplace, to use it day to day. From then on, every headless run that read that person's settings loaded the installed pack as well.

- **In the pack's conditions** this did no harm. Where the lab's copy of a mod and the installed copy share a name, only the lab's copy runs. We checked this rather than assumed it: lsp-plus answered each grep exactly once, before the install and after.
- **In plain Claude Code's condition**, nothing replaced the installed mods. "Plain" was the pack, version 0.4.0. The +2.1% was the pack measured against itself.

The opposite mistake, the same day

Our other harness, for SWE-bench, avoids this problem by leaving the person's settings out entirely. That also left out the one thing every condition is meant to have: the official Python language server plugin, which the person's settings switch on. In the first 45 SWE attempts, lsp-plus had no server to ask and could never answer. We found it by asking the LSP tool directly under the harness's settings: *"No LSP server is configured for .py files."*

A clean environment can remove what a condition needs, just as a dirty one can add what it must not have.

What we changed

- Every run of the click bench now switches the person's installed plugins off explicitly, whatever the condition, and every SWE run switches the official Python language server on explicitly.
- We checked the fix the way we caught the fault, by what loaded. A plain run shows no mod tools at all, and needed 14 requests for the feature, as plain did before the install.
- The contaminated runs are kept, labelled for what they were (`results-v-1015-installedpack.jsonl`): the installed pack 0.4.0 against the lab's pack, the same thing measured twice, a few percent apart. The 45 SWE attempts without a language server are kept apart too.

With plain Claude Code really plain, today's numbers are back where the release measured them, and in a long session larger:

The pack against plain Claude Code, 15 Oct	Change in cost	95% interval
Fresh session, 8 tasks	−5.7%	−9.6% to −1.6%
Long session, 8 tasks (warm runs)	**−37.4%**	−40.9% to −33.9%

"Warm runs" is the other thing a long session can hide. One of the 46 long runs started with 16k tokens in the cache instead of 220k, because its prepared session had been idle for more than an hour, and paid $1.91 to write the cache again for a five-request task. We report the warm runs and say so; the bench records each run's first cache read so that this can be checked.

What we keep from it

- **Name what loads, then check what loaded.** The transcripts say which tools each run had. A five-line check would have caught this on the first run.
- **Distrust good news about the baseline as much as good news about yourself.** "The competition caught up" was as tempting a story as "our tool saves 40%", and it was wrong for a mundane reason.
- **A setting has two sides.** Removing the person's settings removed a plugin the conditions needed. Every exclusion needs the same check as every inclusion.

Method & limits

Click 8.5.0, Opus 5.5, headless Claude Code 2.1.299; plain condition re-run with the installed plugins switched off (four runs per task fresh, two in a long session prepared without any mods). Pack runs from the same day (`results-trail.jsonl`). Data: `results-v-1015.jsonl` ; the bench's switch is `neutral()` in `bench.py` .

Beyond click

Lab notes 9 · 15 Oct 2026 · written by Claude from this repository's data

Every number in the first eight notes comes mostly from one library: click, 79 files of Python, tasks we wrote ourselves. A fair reader's first objection is the obvious one. Maybe the pack is tuned to click. Maybe the code-graph tools would look better on code they were built for. So we ran the main comparisons again, on code we did not choose and tasks we did not write.

Three new grounds

- **A TypeScript library.** chrono, a natural-language date parser (267 TypeScript source files, 753 tests, jest), at release v2.10.3. Eight tasks of the same kinds as click's (trace the callers, rename, explain, look up, list subclasses, a feature to a spec, raise coverage, change a signature), each graded by hidden tests or fixed answers. Every grader was checked both ways: 0 on the untouched code, 1.0 on a reference solution.
- **Real bug reports.** 30 tasks from SWE-bench Verified (Jimenez et al. 2024; OpenAI 2024): issues filed against django, sympy and other Python projects, each stated as the original issue text, graded by the project's own hidden tests. One attempt per task and condition, all four conditions of a task running at the same time.
- **Deployment configuration.** The YAML repository from note 6, small and large.

The pack on TypeScript

64 runs, Opus 5.5, four per task and condition (`results-ts.jsonl`):

	Change in cost, the pack against plain Claude Code	95% interval
All eight tasks	**−7.6%**	−12.1% to −3.1%
Multi-step tasks	−6.3%	−12.5% to +0.8%
Quick questions	**−9.9%**	−13.7% to −5.9%

The pack took fewer requests on all eight tasks: the feature in 10.5 instead of 13.2, the rename in 4.2 instead of 6.0. Quality was the same; each side had one partial answer (plain missed one term of an explanation once, the pack one class of a list once).

One difference from click stands out. On click's quick questions the pack cost 5.5% more on one day and 3.6% less on another, about even (note 1). On TypeScript it saved 9.9% on them. Questions about a typed codebase lean on symbol lookups more, and lsp-plus answers those from the TypeScript server inside the search.

The pack on real bugs

120 attempts, Opus 5.5 at its highest effort, as the harness runs SWE-bench (`swe-bench/2026-10-15-swe`):

Condition	Bugs fixed	Cost of all 30	Requests per bug	Time per bug
Plain Claude Code	29	$15.01	12.9	213 s
The pack	**30**	**$13.17**	**9.9**	**144 s**
The pack with scout and blast	29	$14.13	10.2	150 s
The pack with Trail-Code	29	$14.59	11.5	162 s

With one attempt per bug, cost intervals are wide: the pack against plain −12.3% [−32.5%, +11.9%]. What carries the result is consistency rather than size: on real issues in projects it had never seen, the pack again needed about a quarter fewer requests and finished in two thirds of the time. The one bug only the pack fixed (sympy #22080) is most likely chance at this sample size.

Trail-Code, measured where it reports its own SWE-bench results, again cost more than the pack alone: +10.8% [0.0%, +25.5%]. Scout, the definition pointers from note 4, spoke on 20 of the 30 issues, which name functions and classes far more often than our tasks; it still saved nothing (+7.3% against the pack [−6.7%, +20.7%]). That settles scout: it stays out.

Deployment configuration

Note 6 has the details. On 54 files and on 954 files, Opus and Haiku found every planted broken reference with or without yaml-plus; on the large repository Opus wrote its own reference checker in one step.

What holds, and what does not

- **The pack's saving holds** on a second language, on real bug reports and on code it was not built around: fewer requests at equal quality, everywhere we looked. Its size varies (−6% to −12% fresh, −37% in a long session on click), and with single attempts per bug the SWE interval still crosses zero.
- **Trail-Code cost more than the pack alone on every ground** we measured: click fresh, click long, and SWE-bench.
- **The ideas we borrowed did not survive replication.** Scout's small saving on click did not appear on real issues. yaml-plus found nothing a model did not find itself.
- **The router (route) has not been replicated yet.** Its −17% is from click alone; bug reports are mostly neither features nor tests, so SWE-bench is the wrong ground for it, and a set of real feature requests is the right one.

Method & limits

TypeScript: chrono-node v2.10.3, tasks and graders in `ts_repo.py`; runs with the person's installed plugins switched off and the official TypeScript language server on. SWE-bench: 30 validated SWE-bench Verified tasks, the harness in `pilot/swe.py` of the lab (each condition loads frozen copies of the mods; the official Python language server switched on in every condition, see note 8); one attempt per task and condition, so per-task differences are noise and only totals carry weight. All costs at list prices from transcripts.

Sources

- Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*.
- OpenAI (2024). Introducing SWE-bench Verified.
- chrono, natural-language date parser, release v2.10.3.